

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería
Inteligencia Artificial
Semestre I – 2026



Proyecto 3: Juegos Adversarios

Integrantes:

Nina Nájera
Alejandro Antón
Wilson Calderón

1. Introducción

Este proyecto implementa un agente inteligente para jugar Othello, un juego de suma cero con información perfecta. El objetivo es diseñar un motor capaz de tomar decisiones competitivas dentro de un límite estricto de dos segundos por jugada, comparando el rendimiento de distintos algoritmos de búsqueda adversaria en un entorno de alta complejidad combinatoria.

Se eligió Othello sobre Damas por la naturaleza de sus heurísticas, en especial el control de esquinas y la movilidad, que permiten evaluar con mayor claridad las diferencias entre algoritmos en distintas fases del juego.

2. Arquitectura del sistema

El proyecto está dividido en dos componentes principales que no comparten lógica entre sí.

2.1. GameEngine

Contiene toda la lógica del juego: el estado del tablero, la generación de movimientos legales, la aplicación de jugadas y los algoritmos de búsqueda. Sus métodos principales son:

- `get_legal_moves`: retorna los movimientos válidos para el jugador en turno.
- `evaluate`: función heurística combinada, descrita en la sección siguiente.
- `alpha_beta`: Minimax con poda alfa-beta e `iterative deepening`.
- `expectimax`: variante que modela al oponente como un agente sub-óptimo.
- `mcts`: búsqueda por Monte Carlo con la fórmula UCT.

2.2. GameVisualizer

Interfaz gráfica construida con Pygame. Permite jugar en tres modos: Humano vs Humano, Humano vs IA, e IA vs IA. Muestra en tiempo real los nodos explorados, el tiempo de la última jugada y la evaluación del tablero. Desde el modo IA vs IA se puede lanzar directamente el torneo de 20 partidas.

3. Heurísticas de evaluación

A diferencia de juegos como Tic-Tac-Toe, en Othello no es viable llegar al estado terminal en la mayoría de las búsquedas. Por eso la función de evaluación es un componente crítico del motor.

La función combina cuatro métricas normalizadas:

1. **Paridad de fichas**: diferencia porcentual entre las fichas propias y las del oponente.

2. **Movilidad:** diferencia en la cantidad de movimientos legales disponibles. Tener más opciones que el oponente es una ventaja estructural importante.
3. **Control de esquinas:** las esquinas son las posiciones más estables del tablero porque no pueden ser volteadas. Se penaliza perderlas y se premia capturarlas.
4. **Peso posicional:** cada casilla tiene un valor fijo en una tabla de 8×8 . Las esquinas valen 100, las posiciones adyacentes a ellas valen -20 o -50, y las casillas centrales tienen valores intermedios positivos.

Los pesos de cada componente cambian según la fase del juego:

Fase	Movilidad	Esquinas	Posición	Paridad
Apertura	5.0	8.0	2.0	0.5
Juego medio	3.0	10.0	3.0	1.5
Final	1.0	8.0	1.0	5.0

Cuadro 1: Pesos de la función heurística por fase del juego.

La fase se determina por la cantidad de casillas vacías: apertura con 44 o más, final con 14 o menos, y juego medio en el rango intermedio.

4. Algoritmos de búsqueda

4.1. Alpha-Beta Minimax

Se implementó Minimax con poda alfa-beta e iterative deepening. La búsqueda itera desde profundidad 1 hasta la profundidad configurada, guardando el mejor movimiento encontrado en cada iteración. Si el tiempo restante supera el 85 % del límite, se detiene antes de iniciar la siguiente iteración.

Para mejorar la eficiencia de la poda, los movimientos se ordenan antes de explorar: las esquinas se evalúan primero, luego las posiciones con peso positivo y finalmente las negativas. Esto aumenta la probabilidad de que los cortes beta ocurran temprano.

4.2. Expectimax

Variante del Minimax donde los nodos del oponente no minimizan, sino que calculan el promedio de todos sus movimientos posibles con probabilidad uniforme. Esto modela un oponente que no siempre elige la jugada óptima. Es más lento que Alpha-Beta a igual profundidad porque no puede podar, por lo que se usa con profundidades menores.

4.3. Monte Carlo Tree Search con UCT

MCTS construye el árbol de búsqueda de forma probabilística. Cada iteración selecciona un movimiento raíz usando la fórmula UCT:

$$\text{Score} = \bar{w} + C \cdot \sqrt{\frac{\ln N}{n}} \quad (1)$$

donde $\bar{w}$ es la tasa de victorias acumulada del nodo, N es el total de visitas del padre y n es el número de visitas del nodo. El valor de $C = \sqrt{2}$ balancea exploración y explotación.

Los rollouts usan una política semi-greedy: si hay esquinas disponibles entre los movimientos legales, se elige una de ellas; de lo contrario, se elige de forma aleatoria. El resultado del rollout se retropropaga al nodo raíz correspondiente.

5. Análisis de rendimiento

5.1. Explosión combinatoria

Se midió la cantidad de nodos visitados por Minimax puro y por Alpha-Beta a profundidades 1 a 4 desde el estado inicial del tablero.

Profundidad	Nodos Minimax	b_{eff} Minimax	Nodos Alpha-Beta	b_{eff} Alpha-Beta
1	5	5.00	4	4.00
2	17	4.12	16	4.00
3	73	4.18	65	4.02
4	317	4.22	195	3.74

Cuadro 2: Comparación de nodos visitados y factor de ramificación efectivo.

El factor de ramificación efectivo se calcula como:

$$b_{eff} = \sqrt[d]{n} \quad (2)$$

donde d es la profundidad y n el total de nodos visitados. A profundidad 4, Alpha-Beta reduce el factor de 4.22 a 3.74, lo que implica explorar 195 nodos frente a los 317 de Minimax puro, una reducción del 38 %.

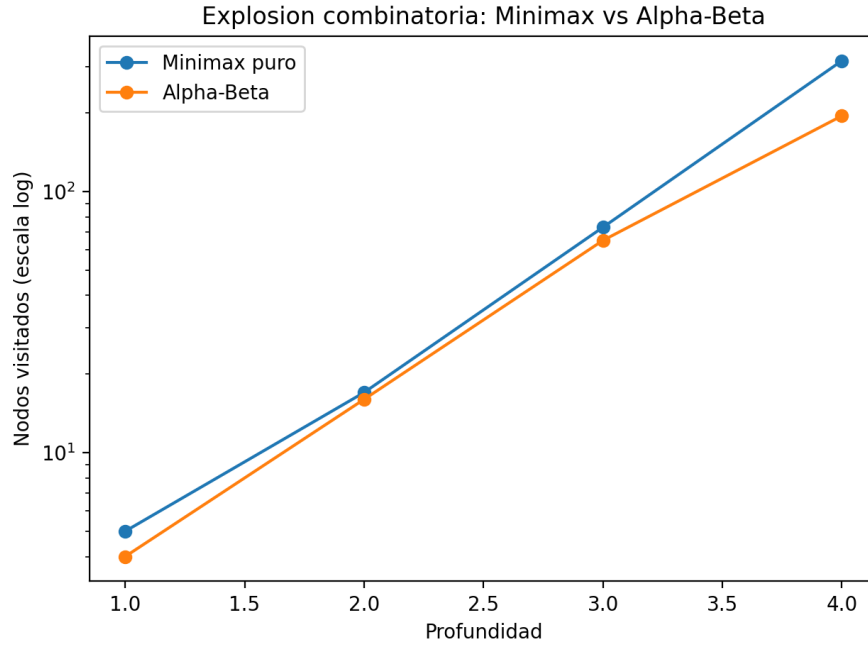


Figura 1: Crecimiento de nodos visitados en escala logarítmica.

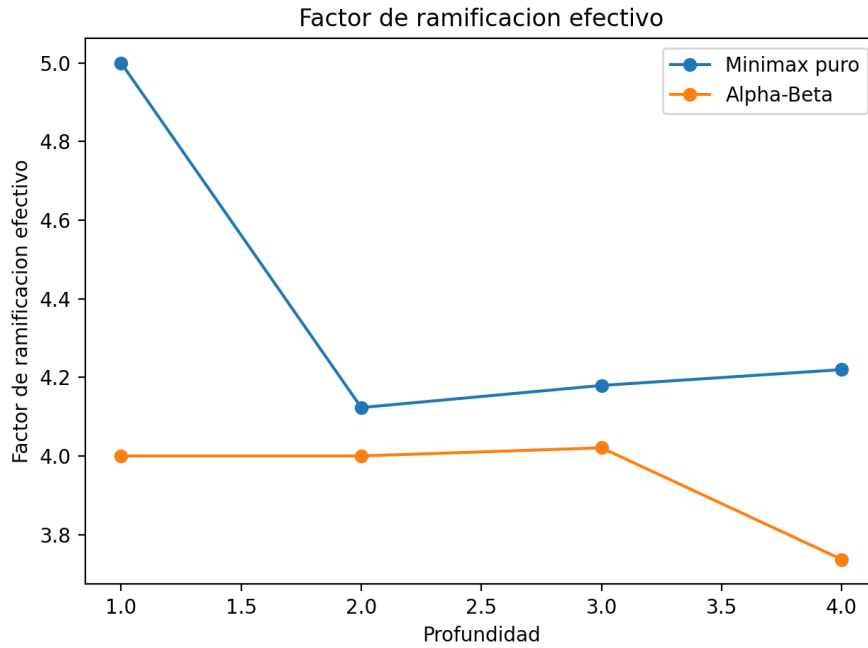


Figura 2: Factor de ramificación efectivo por profundidad.

5.2. Torneo IA vs IA

Se realizaron 20 partidas entre Alpha-Beta con profundidad 6 y MCTS con 1200 iteraciones, alternando colores entre partidas para reducir el sesgo por turno inicial. El límite fue de 2 segundos por jugada.

Agente	Victorias	Tiempo promedio por jugada	Nodos promedio por jugada
Alpha-Beta	16	1.38 s	10,621
MCTS	4	1.32 s	895
Empates	0	–	–

Cuadro 3: Resultados del torneo de 20 partidas.

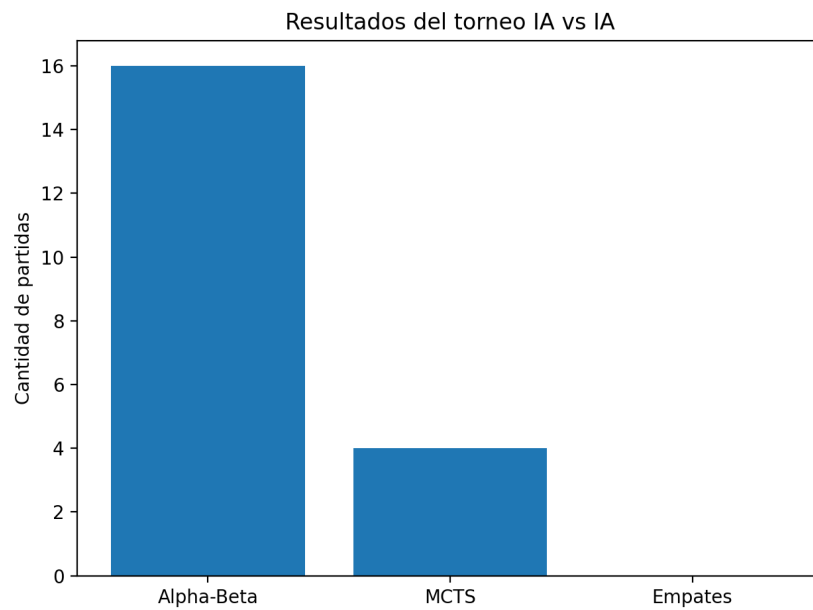


Figura 3: Victorias por agente en el torneo.

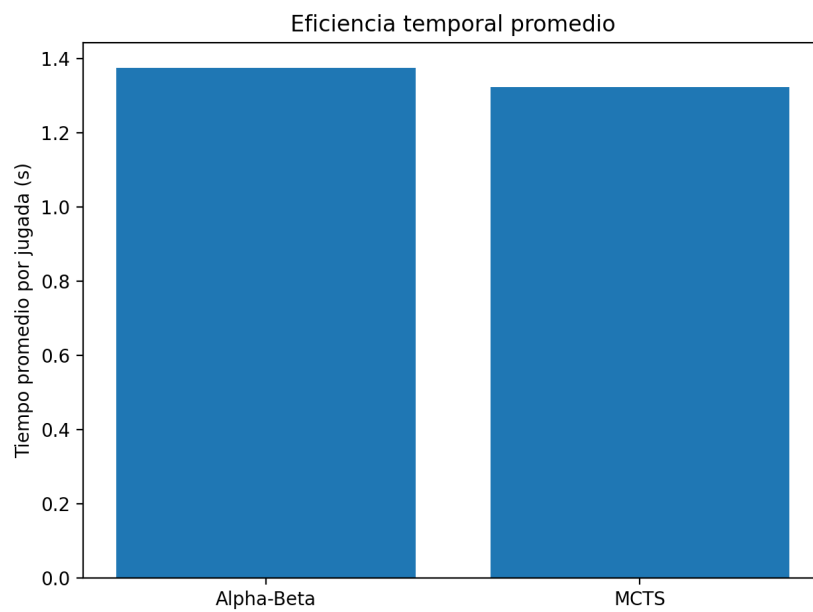


Figura 4: Tiempo promedio por jugada en segundos.

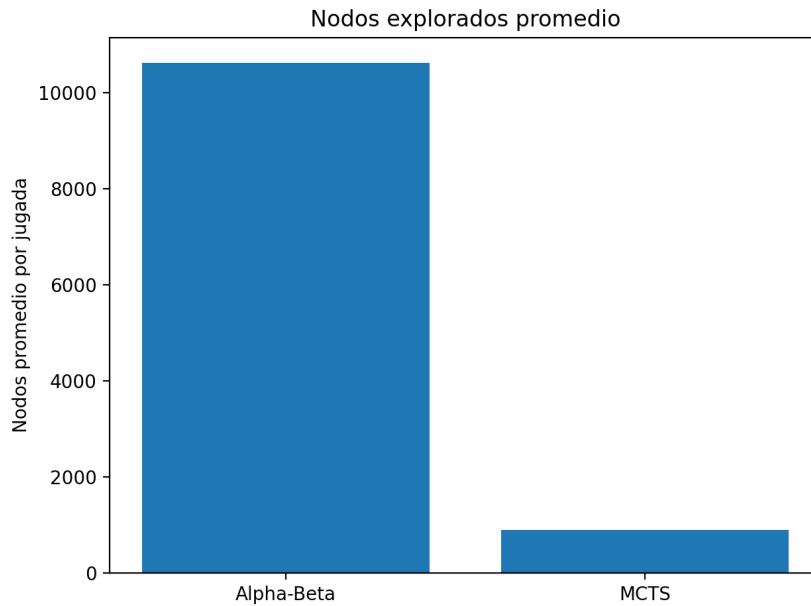


Figura 5: Nodos explorados en promedio por jugada.

Alpha-Beta ganó 16 de las 20 partidas. Aunque los tiempos por jugada son similares, Alpha-Beta explora en promedio 10,621 nodos frente a los 895 de MCTS. Esto se debe a que Alpha-Beta recorre un árbol de búsqueda explícito mientras que MCTS muestrea mediante simulaciones, generando mucho menos estado por iteración.

La ventaja de Alpha-Beta en este torneo se explica principalmente por la calidad de su heurística: al tener una función de evaluación bien calibrada para Othello, la búsqueda determinista con poda supera a MCTS en el presupuesto de tiempo dado. MCTS requiere más simulaciones para converger a jugadas de igual calidad, especialmente en la fase de apertura donde el árbol es amplio.

6. Conclusiones

Alpha-Beta con heurística combinada y ordenamiento de movimientos es el agente más competitivo bajo el límite de 2 segundos. La poda reduce sustancialmente el espacio explorado conforme aumenta la profundidad, y el ajuste de pesos por fase permite al agente priorizar movilidad en la apertura y paridad en el cierre.

MCTS es más flexible porque no depende de una heurística de evaluación explícita, pero necesita un presupuesto de simulaciones mayor para igualar la calidad de Alpha-Beta en Othello. Con más tiempo o más iteraciones, la diferencia entre ambos se reduciría.

Expectimax resultó útil para partidas contra jugadores humanos porque su modelado de oponente sub-óptimo es más realista en ese contexto, aunque es el algoritmo más lento de los tres a igual profundidad.